

**YENEPOYA INSTITUTE OF ARTS, SCIENCE,
COMMERCE AND MANAGEMENT**

A Constituent Unit of Yenepoya (Deemed to be University)
Balmatta, Mangalore

Final Project Report
on
Identity Security Framework for Supply Chain

Submitted by

Alvia Dsouza

23BCCDD024

BCA (Cybersecurity, Digital Forensics and Data Science)

Under the Guidance of

Ms. Divya N Shetty

Lecturer, Department of Computer Science



YENEPOYA

(DEEMED TO BE UNIVERSITY)

Recognized under Sec 3(A) of the UGC Act 1956

Accredited by NAAC with 'A' Grade

Department of Computer Science

Yenepoya Institute of Arts, Science, Commerce and Management

April 2026

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to all those who have supported and guided me throughout the development of this project.

I am deeply grateful to Dr. Jeevan Raj, Principal of Yenepoya Institute of Arts, Science, Commerce and Management for providing the academic infrastructure and opportunities that made this work possible.

I extend my heartfelt thanks to Mr. Rathnakar Shetty, Head of the Department of Computer Science for the constant support and encouragement extended to all students in pursuing their final year projects.

I am especially thankful to my internal project guide, Ms. Divya N Shetty, Lecturer, Department of Computer Science, for her patient guidance, constructive feedback, and continuous encouragement throughout every phase of this project. Her insights into web application security and software design were invaluable in shaping the direction of this work.

I also sincerely thank my industry mentor, Mr. Shashank, for providing real-world perspective and industry-relevant guidance that helped align this project with practical cybersecurity standards.

Lastly, I thank my faculty mentors, family and friends for their unwavering support and encouragement throughout this journey.

Alvia Dsouza

April 2026

TABLE OF CONTENTS

Sl.no	Content	Page
	Executive Summary	1
1	Background	2
2	Aim	4
3	Technologies	6
4	Hardware Architecture	9
5	Software Architecture	10
6	System Requirements	14
6.1	Functional Requirements	14
6.2	User Requirements	15
6.3	Environmental Requirements	15
7	Design and Architecture	17
8	Implementation	23
9	Testing	27
9.1	GUI Layout	27
9.2	Customer Testing	30
10	Evaluation	35
11	Snapshots of the Project	39
12	Conclusions	47
13	Further Development	49
14	References	51
15	Appendix	52

EXECUTIVE SUMMARY

The Identity Security Framework for Supply Chain is a web-based authentication and authorization system developed using Python, Flask, Flask-Login, SQLAlchemy, and SQLite. The project was undertaken as a final year academic project under the IBM Innovation Centre for Education programme, with the primary objective of building a functional, secure login system that demonstrates fundamental cybersecurity concepts in a real-world supply chain context.

Background: Supply chain systems involve multiple categories of stakeholders — administrators, suppliers, and distributors — each requiring different levels of access to shared digital platforms. Most small and medium-scale supply chain applications lack even basic security measures such as password hashing and role-based access control. This project addresses that gap by building a lightweight identity management system from scratch.

Aim: To design and develop a secure web application that implements user registration with hashed password storage, session-based authentication using Flask-Login, and role-based authorization that differentiates between Admin, Supplier, and Distributor users. The project demonstrates the fundamental difference between Authentication (verifying who a user is) and Authorization (controlling what a user can do).

Technologies: Python 3.x, Flask, Flask-Login, Flask-SQLAlchemy, Werkzeug, SQLite, Jinja2, HTML, CSS.

The system was developed following the Waterfall methodology. It was tested through both Black Box and White Box testing techniques and achieved all its defined functional objectives — secure registration, login, session management, role-based access, and admin user management within the defined scope.

1. BACKGROUND

The rapid digitization of supply chain operations has introduced significant cybersecurity challenges. Organizations across the supply chain from raw material suppliers to final distributors are increasingly relying on shared digital platforms for managing inventory, tracking orders, and coordinating logistics. These platforms handle sensitive business data and require controlled access for different types of users.

Despite this growing dependency on digital systems, many small and medium-scale supply chain platforms are developed without adequate security practices. Common vulnerabilities include storing passwords in plain text, using no session management, and implementing no access control that differentiates between user types. These weaknesses expose organizations to threats such as unauthorized access, credential theft, account takeover, and privilege escalation.

At the heart of any secure web application lie two foundational concepts: Authentication and Authorization. Authentication is the process of verifying the identity of a user, confirming that they are who they claim to be, typically through a username and password. Authorization is the process of determining what an authenticated user is permitted to do controlling which pages they can access and which actions they can perform. These two concepts are distinct, and many real-world security vulnerabilities arise from confusing them or implementing either one incorrectly.

This project was conceived to address this gap at an educational level. By building a working authentication and authorization system for a supply chain scenario, the project provides a practical, hands-on demonstration of how these concepts are implemented correctly using industry-standard technologies. The system serves both as a functional prototype and as an educational artifact.

The choice of a supply chain context makes the project directly applicable to real-world scenarios. Supply chain systems naturally involve multiple types of users administrators who manage the platform, suppliers who manage inventory or product data, and distributors who handle logistics making them an ideal setting for

demonstrating role-based access control. Each role has different access privileges, and the system enforces these boundaries at the server side.

The concept of identity management in digital systems has evolved significantly over the past decade. Early web applications relied on simple username and password combinations stored in plain text databases, with no consideration for session security or role differentiation. As cyber threats became more sophisticated, the industry moved toward hashed password storage, encrypted sessions, and granular access control systems. Today, identity and access management (IAM) is considered a foundational discipline in cybersecurity, with dedicated frameworks and standards governing how systems should handle user identities.

In the context of this project, the supply chain serves as a realistic and relatable domain for demonstrating these concepts. A supply chain typically involves three categories of stakeholders with fundamentally different responsibilities. Administrators are responsible for managing the platform, overseeing all users, and maintaining the integrity of the system. Suppliers are responsible for managing raw material inventory, updating stock levels, and communicating with distributors. Distributors are responsible for tracking outbound shipments, managing delivery schedules, and coordinating with suppliers. Each of these roles requires access to different parts of the system, and a well-designed identity framework must ensure that each user can only access what is relevant to their role nothing more and nothing less.

The increasing reliance on web-based platforms has also led to a rise in automated attacks targeting authentication systems. Attackers often use tools to perform credential stuffing, where previously leaked username-password combinations are tested across multiple platforms. In systems where passwords are stored insecurely or where login responses reveal too much information, such attacks can succeed at scale.

Another critical issue is the lack of proper session handling. Insecure session management can allow attackers to hijack active sessions and gain unauthorized access without needing credentials. This highlights the importance of secure session cookies, proper logout mechanisms, and server-side session validation.

2. AIM

The primary aim of this project is to design, develop, and demonstrate a secure identity management system for a supply chain web application. The system is intended to serve as both a functional application and an educational demonstration of core web application security principles.

The specific objectives of the project are as follows:

- To implement a secure user registration system that collects user details including first name, last name, email address, username, password, and role, validates all inputs on the server side, and stores the hashed password in a local database. The password is never stored in plain text at any stage.
- To implement a credential-based login system that retrieves the user record from the database, verifies the submitted password against the stored hash using a secure comparison function, and creates a signed session cookie on successful verification.
- To implement role-based access control that enforces different levels of access for Admin, Supplier, and Distributor users, enforced server-side and not bypassable through client-side manipulation.
- To implement a secure logout mechanism that correctly terminates the user's session and invalidates the session cookie.
- To provide a clear educational demonstration of the difference between Authentication and Authorization through the system's structure and dashboard content.

The project follows the Waterfall software development methodology, which is appropriate for a project of this nature given that the requirements were clearly defined from the outset and the scope was fixed. The development phases progressed sequentially through requirements gathering, system design, implementation, testing, and documentation. This structured approach ensured that each component of the system was fully specified before being built, reducing the likelihood of design changes during implementation. The Waterfall model also

made it straightforward to produce comprehensive documentation at each phase, which aligns with the academic objectives of the project.

3. TECHNOLOGIES

The project uses the following technologies, each selected for a specific reason:

Technology	Role	Rationale
Python 3.x	Programming Language	Readable, strong ecosystem, widely used
Flask	Web Framework	Lightweight, flexible, industry-standard
Flask-Login	Session Management	Purpose-built for Flask, handles lifecycle cleanly
Flask-SQLAlchemy	ORM / Database Access	Abstracts SQL, simplifies interactions
Werkzeug	Password Hashing	PBKDF2-HMAC-SHA256, bundled with Flask
SQLite	Database	Zero-config, file-based, ideal for development
Jinja2	HTML Templating	Bundled with Flask, secure and powerful
HTML / CSS	Frontend	No framework dependency, quick to develop
VS Code	Development Environment	Lightweight, strong Python support

Python 3.x:

Python 3.x was chosen as the primary programming language for its readability, strong web ecosystem, and widespread adoption in both academic and professional settings. Python's clean syntax makes the security logic of each route handler easy to read and understand, which is particularly important for an educational project whose goal is to demonstrate security concepts transparently. The Flask web framework and all supporting libraries are Python packages, making Python the natural foundation for the stack.

Flask:

Flask was selected as the web framework because it is lightweight, flexible, and industry-standard for Python web development. Flask follows a micro-framework philosophy — it provides the core components needed for routing, request handling, and template rendering, and leaves all other decisions to the developer. This is in contrast to heavier frameworks such as Django, which follow a 'batteries included' philosophy and provide built-in authentication systems, admin interfaces, and ORMs.

The deliberate choice of Flask over Django was made because Flask requires the developer to explicitly implement each security feature — password hashing, session management, and access control rather than inheriting them from a pre-built system. This transparency makes Flask the ideal choice for an educational project whose primary goal is to demonstrate how these mechanisms work internally. Every security decision in this project is visible in the code and can be traced directly to its implementation.

Flask-Login:

Flask-Login provides session management specifically designed for Flask applications. It provides the `@login_required` decorator for protecting routes, the `current_user` proxy for accessing the logged-in user's data from any route, the `user_loader` callback for loading the user object from the session on each request, and the `login_user()` and `logout_user()` functions for creating and destroying sessions. Using Flask-Login ensures that session management follows established best practices rather than a custom implementation.

Werkzeug:

Werkzeug was selected for password hashing rather than implementing a custom hashing solution. Writing custom cryptographic code is widely considered a dangerous practice in software development because even small mistakes in implementation can render a hashing algorithm completely ineffective. Werkzeug's `generate_password_hash` function has been audited, tested, and maintained by security

professionals, and implements the PBKDF2-HMAC-SHA256 algorithm with a randomly generated salt and a high iteration count. This makes it both secure and trustworthy for use in a real-world application.

Flask-SQLAlchemy and SQLite:

Flask-SQLAlchemy provides an Object-Relational Mapping layer that allows database interactions to be expressed as Python method calls rather than raw SQL strings. This abstraction improves code readability, prevents SQL injection vulnerabilities, and makes the database backend replaceable without changing application logic. SQLite was chosen as the database because it requires no separate server process, making it ideal for local development and demonstration. The UNIQUE constraints on the username and email columns cause SQLite to create implicit indexes on these fields, making login and duplicate-check queries efficient indexed lookups.

4. HARDWARE ARCHITECTURE

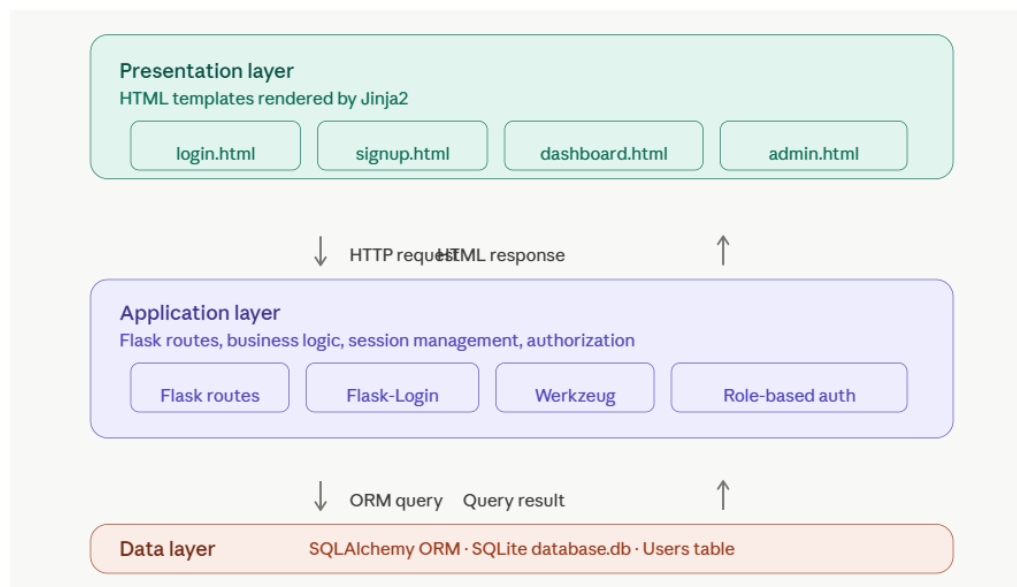
The application is designed for local development and runs on any standard laptop or desktop computer with the following minimum specifications:

- **Processor:** Any modern processor (Intel Core i3 or equivalent and above) capable of running Python 3.x. Flask's development server and SQLite impose negligible CPU overhead for a single user-application.
- **RAM:** Minimum 4 GB. Flask's development server and SQLite impose negligible memory overhead for a single-user application. 8 GB is recommended for comfortable simultaneous use of the browser, VS Code, and the Flask server.
- **Storage:** Minimum 500 MB of free disk space for Python, Flask, its dependencies, and the SQLite database file. The database.db file itself will be only a few kilobytes for the expected number of test users.
- **Operating System:** Any OS supporting Python 3.x — Windows 10/11, macOS 10.15+, or Ubuntu 20.04+
- **Network:** A stable local connection for loading Google Fonts in the UI. The application runs entirely on localhost.
- **Browser:** Any modern web browser — Google Chrome, Mozilla Firefox, or Microsoft Edge capable of rendering CSS animations, CSS custom properties, and ES6 JavaScript.

No specialized hardware such as a dedicated server, GPU, or network appliance is required. The entire application stack runs on the developer's local machine, making it fully self-contained and portable, and reproducible on any standard development workstation.

5. SOFTWARE ARCHITECTURE

The application follows a three-layer software architecture, which separates the system into distinct layers each with a specific responsibility. This separation of concerns makes the codebase maintainable, testable, and extensible, and ensures that the security logic in the application layer cannot be bypassed by manipulating the presentation layer.



5.1 Presentation Layer

The topmost layer consists of four HTML templates rendered by the Jinja2 templating engine: login.html, signup.html, dashboard.html, and admin.html. This layer is responsible for all user interaction — displaying forms, receiving input, and presenting information to the user. Data flows from this layer to the application layer through HTTP POST requests submitted via HTML forms. Responses flow back from the application layer as rendered HTML pages or HTTP redirects.

5.2 Application Layer

The middle layer is the Flask web application defined in `app.py`. This layer contains all the business logic — URL routing, form input extraction and validation, password hashing, session creation and termination, role-based authorization checks, and database interaction through SQLAlchemy. It acts as the bridge between the presentation layer and the data layer. Flask-Login operates within this layer to manage user authentication state across requests.

All security-critical decisions are made in this layer. Input validation, password hashing, session creation, and role checks all happen here, on the server side, where they cannot be bypassed by client-side manipulation. The application layer returns either a rendered HTML page (via `render_template()`) or an HTTP redirect (via `redirect()`) depending on the outcome of each request.

5.3 Data Layer

The bottom layer consists of the SQLite database file (`database.db`) accessed through Flask-SQLAlchemy. The data model is defined in `models.py` as the `User` class, which maps directly to the `users` table in the database. All database operations — inserting, reading, and deleting user records — are performed through SQLAlchemy's ORM without any raw SQL. This ensures that the application is protected against SQL injection by construction, and that the database backend can be replaced without changing any application logic.

5.4 File Structure

The application is organized into the following files:

- `app.py` — Main application file containing Flask instance, configuration, and all route handlers

- `models.py` — Database model definition containing the User class that maps to the users table in SQLite.
- `templates/login.html` — The login page template with username and password form.
- `templates/signup.html` — The registration page template with all six input fields and role selection cards.
- `templates/dashboard.html` — The user dashboard template with role-specific content rendering.
- `templates/admin.html` — The admin panel template with the user management table.
- `instance/database.db` — The SQLite database file, automatically generated on first application startup.

5.5 System Request-Response Flow

Every user interaction in the system follows a structured request-response cycle. When a user submits a form, such as during login or registration, the browser sends an HTTP POST request to the Flask server. The server processes the request by validating inputs, performing necessary database operations, and determining the appropriate response.

If the request is valid, the server either renders a new HTML page or redirects the user to another route. If the request fails validation, the server sends back the same page along with an appropriate flash message. This cycle ensures that all critical logic remains on the server side. A user cannot bypass validation by manipulating the HTML form fields in the browser, because the server re-validates all inputs independently regardless of what the client sends.

5.6 Development Methodology

The project was developed following the Waterfall methodology, with the development process divided into five sequential phases. The first phase was requirements analysis, during which the functional, user, and environmental requirements of the system were defined. The second phase was system design, during which the database schema, route

structure, security architecture, and UI layout were planned before any code was written. The third phase was implementation, during which the Flask application, data model, route handlers, and HTML templates were built. The fourth phase was testing, during which the system was tested manually using both Black Box and White Box techniques across unit, integration, and system testing levels. The fifth phase was documentation, during which the project synopsis, low level design, high level design, and final report were prepared.

This sequential approach was appropriate for this project because the requirements were well-defined and stable from the beginning, and the scope was small enough that iterative development was not necessary. Each phase produced a clear deliverable that fed directly into the next phase, making the overall development process orderly and traceable.

6. SYSTEM REQUIREMENTS

6.1 Functional Requirements

The system must allow a new user to register by submitting a form containing their first name, last name, email address, username, password, and selected role. All submitted fields must be validated on the server side. Specifically: all fields must be present; username must be a minimum of four characters; password must be a minimum of six characters; role must be admin, supplier, or distributor; username must be unique; email must be unique.

The system must hash the user's password using Werkzeug's PBKDF2-HMAC-SHA256 algorithm before storing it. Plain text passwords must never be stored or logged.

The system must allow a registered user to log in by verifying their password against the stored hash and creating a signed session cookie on success.

The system must protect all sensitive pages using Flask-Login's `@login_required` decorator. Unauthenticated requests must be redirected to the login page.

The system must ensure that all sensitive operations are handled securely and must not expose internal implementation details to the user. Error messages should be generic to prevent attackers from gaining insights into system behaviour.

The system must enforce role-based access control. Admin users access the admin panel and have full system oversight. Supplier and Distributor users are restricted to their role-specific dashboard only, where they can see only the features relevant to their role.

The system must provide a logout function that terminates the session and redirects to login. The admin panel must allow viewing and deletion of user accounts, with a guard preventing self-deletion.

6.2 User Requirements

The user requirements define how the system must behave from the perspective of different user types.

- New users must be able to register without any technical knowledge. The registration form must be clearly labeled with descriptive placeholder text and helpful error messages when validation fails.
- Returning users must be able to log in quickly and receive clear, immediate feedback on failure. The login form must not reveal whether the username or the password was wrong.
- Authenticated users of all roles must see their name, email, and role clearly displayed on the dashboard after login.
- Admin users must be able to view all registered users and delete accounts with a single click from the admin panel.
- All users must experience a consistent and intuitive interface across all pages, with clear navigation options and no dead ends.
- The system should provide a responsive interface that works correctly on standard desktop and laptop screen sizes.

6.3 Environmental Requirements

The application requires the following software environment to run:

- Python 3.x installed on the development machine.
- Required packages installed via pip: flask, flask-login, flask-sqlalchemy. Werkzeug and Jinja2 are installed automatically as Flask dependencies.
- Installation command: `pip install flask flask-login flask-sqlalchemy`

- Run command: `python app.py`
- Access: `http://127.0.0.1:5000/`
- Testing browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

No web server, database server, or cloud infrastructure is required. The application runs entirely on the developer's local machine.

7. DESIGN AND ARCHITECTURE

7.1 System Design Overview

The Identity Security Framework is designed around the principle of security by default. Every design decision from the password hashing mechanism to the session cookie signing to the server-side role checks prioritizes the protection of user data and the enforcement of access boundaries. The system is built to resist the most common web application attacks including credential stuffing, rainbow table attacks, session hijacking, and privilege escalation, within the scope of its defined requirements.

The system follows a three-layer architecture as described in the Software Architecture section. All security-critical operations occur in the application layer and cannot be bypassed by manipulating the presentation layer. The data layer stores only hashed passwords and enforces uniqueness constraints at the database level, providing a second layer of protection even if the application layer were somehow bypassed.

7.2 Database Design

The system uses a single SQLite database with one table — users. The table is created automatically by SQLAlchemy on the first application startup. The schema is as follows:

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user identifier
first_name	VARCHAR(100)	NOT NULL	User's first name
last_name	VARCHAR(100)	NOT NULL	User's last name
email	VARCHAR(150)	NOT NULL, UNIQUE	Email, stored lowercase
username	VARCHAR(100)	NOT NULL, UNIQUE	Username, min 4 chars
password	VARCHAR(200)	NOT NULL	Werkzeug PBKDF2 hash
role	VARCHAR(50)	NOT NULL	admin / supplier / distributor

The password column stores the full Werkzeug hash string in the format `pbkdf2:sha256:<iterations>$<salt>$<hash>`. This string is self-contained it carries all information needed for future verification. Plain text passwords are never written to this column.

The database design ensures data integrity through the use of constraints such as PRIMARY KEY and UNIQUE. These constraints prevent duplicate records and maintain consistency. Additionally, storing hashed passwords instead of plain text significantly enhances data security.

7.3 Route Design

The application defines seven routes as shown in the table below. Each route in the application follows RESTful design principles, where each URL represents a specific resource or action. The use of GET and POST methods ensures clear separation between data retrieval and data submission operations.

Route	Method	Auth Required	Role	Purpose
/	GET	No	None	Redirect to /login
/signup	GET, POST	No	None	User registration
/login	GET, POST	No	None	User authentication
/dashboard	GET	Yes	Any	User dashboard
/logout	GET	Yes	Any	Session termination
/admin	GET	Yes	Admin	Admin user management
/delete/<id>	GET	Yes	Admin	Delete user by ID

7.4 Security Design

The security architecture is built on four pillars:

- Password hashing - PBKDF2-HMAC-SHA256 with a random salt ensures identical passwords produce different hash values, protecting against rainbow table attacks.
- Session security - Flask signs session cookies using HMAC with the SECRET_KEY, preventing forgery.
- Server-side access control - @login required and role checks are enforced before any route handler executes.
- Input validation - all form data is validated server-side before any database operation.

The system adopts a defense-in-depth strategy by implementing multiple layers of security controls. Even if one mechanism fails, others remain in place to protect the system.

Security Threat Analysis: To further strengthen the security design, a threat analysis was performed to identify potential vulnerabilities and the corresponding mitigation techniques implemented in the system. This analysis ensures that common web application attacks are anticipated and prevented through appropriate design decisions.

Threat	Description	Mitigation
Brute Force	Multiple login attempts	Password hashing
SQL Injection	Malicious queries	ORM usage
Session Hijacking	Cookie theft	Signed sessions
XSS	Script injection	Jinja2 escaping

The table above highlights some of the most common security threats faced by web applications and the specific measures implemented in this project to mitigate them. Password hashing using PBKDF2 significantly reduces the risk of brute force attacks

by increasing computational effort. SQL injection risks are minimized through the use of SQLAlchemy ORM, which avoids raw SQL queries. Session hijacking is mitigated by using signed session cookies, ensuring that session data cannot be tampered with. Cross-site scripting (XSS) attacks are prevented through Jinja2's automatic escaping of user inputs in templates.

This layered approach to security aligns with industry best practices and ensures that the system remains resilient against multiple types of attacks.

7.5 User Interface Design

The user interface of the application was designed with a consistent cybersecurity-themed visual language across all four pages. The design uses a dark background color (#050a0f) that evokes the aesthetic of professional security tools and terminal interfaces. The primary accent color is a teal (#00dca8) that is used for headings, labels, borders, and interactive elements. Error states use a coral red (#ff4d6d) for flash messages and validation indicators. All text uses the Share Tech Mono monospace font for labels and technical elements, and the Rajdhani sans-serif font for headings and body content.

Each page features an animated grid background that moves slowly, creating a sense of depth and activity without distracting from the content. A radial glow effect centered on the page reinforces the cybersecurity aesthetic. Floating particle animations rise from the bottom of the screen on the login and signup pages. Corner bracket decorations on the main card elements are a common design pattern in cybersecurity interfaces and reinforce the theme consistently.

The signup page was designed with first name and last name fields placed side by side on the same row using a CSS grid layout, with the remaining fields stacked vertically below. Role selection is implemented as styled radio button cards rather than a dropdown, making the selection more visual and intuitive. The admin panel uses a table layout with teal-tinted column headers, alternating row shading, and red delete buttons that are visually distinct and clearly actionable. A status bar fixed at the bottom of the

login and signup pages displays system status information that reinforces the security theme.

7.6 Waterfall Development Phases

Phase 1 - Requirements Analysis: The functional requirements were defined based on the project brief provided by the IBM Innovation Centre for Education. The core requirement was to build a login system demonstrating authentication and authorization for a supply chain context. From this, the specific requirements were derived — three user roles, server-side validation, password hashing, session management, and an admin panel.

Phase 2 - System Design: The database schema was designed as a single users table with seven columns. The seven application routes were planned with their HTTP methods, authentication requirements, and role constraints. The three-layer architecture was defined with clear boundaries between the presentation, application, and data layers. The security design was planned around four pillars - password hashing, session security, server-side access control, and input validation.

Phase 3 - Implementation: The models.py file was created first to define the data model. The app.py file was then built incrementally starting with the Flask configuration and database initialization, then adding routes one by one in the order: signup, login, dashboard, logout, admin, delete. The HTML templates were developed alongside their corresponding routes. CSS styling was applied after the functional structure was complete.

Phase 4 - Testing: All 14 test cases were executed manually using Google Chrome. Each test case was designed to verify a specific functional requirement - form validation, password verification, session management, role-based access, or admin functionality.

Phase 5 - Documentation: The project synopsis, LLD, HLD, and final report were prepared sequentially, with each document building on the work documented in the previous one.

8. IMPLEMENTATION

8.1 Development Environment Setup

Python 3.x was installed on the development machine. A project folder was created and required packages were installed using `pip install flask flask-login flask-sqlalchemy`. Project files were created and managed in VS Code. The application was tested using Google Chrome.

8.2 User Model Implementation

The User model is defined in `models.py`. The User class inherits from both `db.Model` (Flask-SQLAlchemy) and `UserMixin` (Flask-Login). The `db.Model` inheritance causes SQLAlchemy to map the class to the users database table. The `UserMixin` inheritance provides the four default properties required by Flask-Login: `is_authenticated`, `is_active`, `is_anonymous`, and `get_id()` without requiring manual implementation.

8.3 Application Configuration

The Flask application is configured with three key settings: `SECRET_KEY` for signing session cookies, `SQLALCHEMY_DATABASE_URI` pointing to the SQLite file, and `SQLALCHEMY_TRACK_MODIFICATIONS` set to `False`. Flask-Login is configured by setting `login_manager.login_view = 'login'`. The `user_loader` callback returns the User object using `db.session.get(User, int(user_id))`.

8.4 Registration Route

The signup route handles GET and POST requests. On POST, it extracts six form fields, strips whitespace, lowercases the email, and runs a 7-step validation sequence: all fields present, username at least 4 characters, password at least 6 characters, role valid, username unique, email unique. If all checks pass, the password is hashed using `generate_password_hash()`, a new User object is created, added to `db.session`, committed, and the user is redirected to login with a success message.

8.5 Login Route

On POST, the login route queries the database using `User.query.filter_by(username=username).first()`. If a user is found and `check_password_hash(user.password, password)` returns True, `login_user(user)` is called and the user is redirected to the dashboard. If either check fails, the same generic error 'Invalid username or password.' is flashed regardless of which check failed, preventing username enumeration attacks.

8.6 Admin and Delete Routes

The admin route checks `current_user.role == 'admin'`. Only admin users can access this route. Otherwise it calls `User.query.all()` and renders `admin.html`. The delete route performs the same role check, then fetches the target user by ID, compares it against `current_user.id` to prevent self-deletion, and if valid calls `db.session.delete(user)` and `db.session.commit()`.

8.7 Template Implementation

All four templates use Jinja2 syntax for dynamic content. Flash messages are rendered using `get_flashed_messages(with_categories=true)`. The dashboard template checks `current_user.role` to conditionally render the admin panel link. The admin template uses a for loop to generate one table row per registered user. All templates share a consistent dark cybersecurity-themed CSS design.

8.8 Role-Specific Dashboard Implementation

The dashboard is the most complex template in the application because it renders different content based on the logged-in user's role. The `dashboard.html` template uses Jinja2's conditional rendering to display role-specific panels. When the logged-in user has the Admin role, the dashboard displays the Admin Control Panel section with a

FULL ACCESS badge, a View All Users button that links to /admin, and four feature cards : User Management, System Monitoring, Security Config, and Role Assignment all marked as GRANTED.

When the logged-in user has the Supplier role, the dashboard displays the Supplier Operations Panel with a LIMITED ACCESS badge and four feature cards :Inventory Management (GRANTED), Shipment Tracking (GRANTED), User Management (DENIED marked in red to indicate this is an admin-only operation), and Distributor Communications (GRANTED). This visual DENIED indicator is a deliberate design choice that makes the concept of Authorization immediately visible and understandable the same system, the same page, but completely different content and access levels based on the user's assigned role.

When the logged-in user has the Distributor role, the dashboard similarly displays a Distributor Operations Panel with role-appropriate content. The navbar at the top of the dashboard displays the logged-in user's username and role, a live clock, and a Logout button, providing a consistent and informative header across all sessions.

8.9 Flash Message System

Flask's flash messaging system is used throughout the application to communicate success and error states to the user. Flash messages are stored in the session after being added with the flash() function and are consumed and cleared when get_flashed_messages() is called in the next rendered template. This means a flash message added in a POST route handler will be displayed on the next page the user sees after the redirect.

The application uses two flash message categories - 'error' and 'success'. Error messages are displayed with a red border and red text, using the styling defined in each template's CSS. Success messages are displayed with a teal border and teal text. The login page,

signup page, and admin panel all include the Jinja2 flash message block that iterates over all pending messages and renders them above the main content area.

Flash messages are used in the following situations: when any signup validation check fails (error), when an account is created successfully (success), when login credentials are invalid (error), when a user is logged out (success), when an admin tries to delete their own account (error), and when a user is deleted successfully (success).

9. TESTING

9.1 Graphical User Interface (GUI) Layout

The application consists of four HTML pages, each with a consistent dark cybersecurity-themed design. Screenshots of each page are shown below.

Login Page (login.html):

The login page presents a dark-themed interface with the heading IDENTITY VERIFICATION and subtitle SUPPLY CHAIN SECURITY FRAMEWORK v1.0. It contains username and password input fields and a teal AUTHENTICATE button. Flash messages appear above the form.

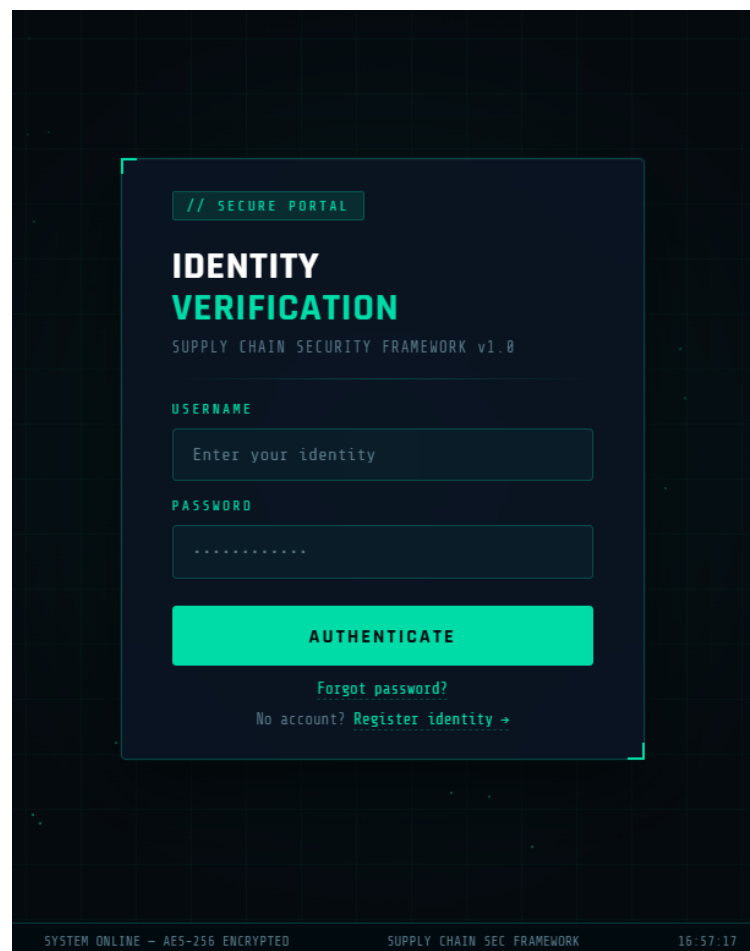


Figure 1: Login Page — login.html

Signup Page (signup.html):

The signup page presents the heading REGISTER ACCESS. It contains input fields for first name, last name, email, username, and password, followed by three styled radio button role cards : Admin, Supplier, and Distributor.

The screenshot displays a dark-themed web interface for a supply chain identity registration system. At the top left, a teal button labeled '// NEW IDENTITY' is visible. The main heading is 'REGISTER ACCESS' in large, bold, teal letters, with the subtitle 'CREATE YOUR SUPPLY CHAIN IDENTITY' below it. The form consists of several input fields: 'FIRST NAME' and 'LAST NAME' (each with a placeholder 'Enter first name' and 'Enter last name'), 'EMAIL ADDRESS' (placeholder 'Enter your email'), 'USERNAME' (placeholder 'Choose your identity'), and 'PASSWORD' (placeholder '.....'). Below these fields is the 'ASSIGN ROLE' section, which contains three radio button options: 'ADMIN' (with a shield icon), 'SUPPLIER' (with a box icon), and 'DISTRIBUTOR' (with a truck icon). A large teal button labeled 'REGISTER IDENTITY' is positioned below the role selection. At the bottom of the form, there is a link: 'Already registered? [Authenticate →](#)'. The footer of the page shows system status: '● SYSTEM ONLINE - BCrypt HASHING ACTIVE', 'SUPPLY CHAIN SEC FRAMEWORK', and the time '16:59:52'.

Figure 2: Signup Page — signup.html

Dashboard (dashboard.html):

The dashboard displays the logged-in user's name, email, and role. It contains two educational concept cards explaining Authentication and Authorization. An admin panel link is conditionally visible only for Admin users. The role-specific operations panel below changes based on the user's role — Admin users see the Admin Control Panel with FULL ACCESS, Supplier/Distributor users see their Operations Panel with LIMITED ACCESS. The admin panel link is visible only to Admin users.

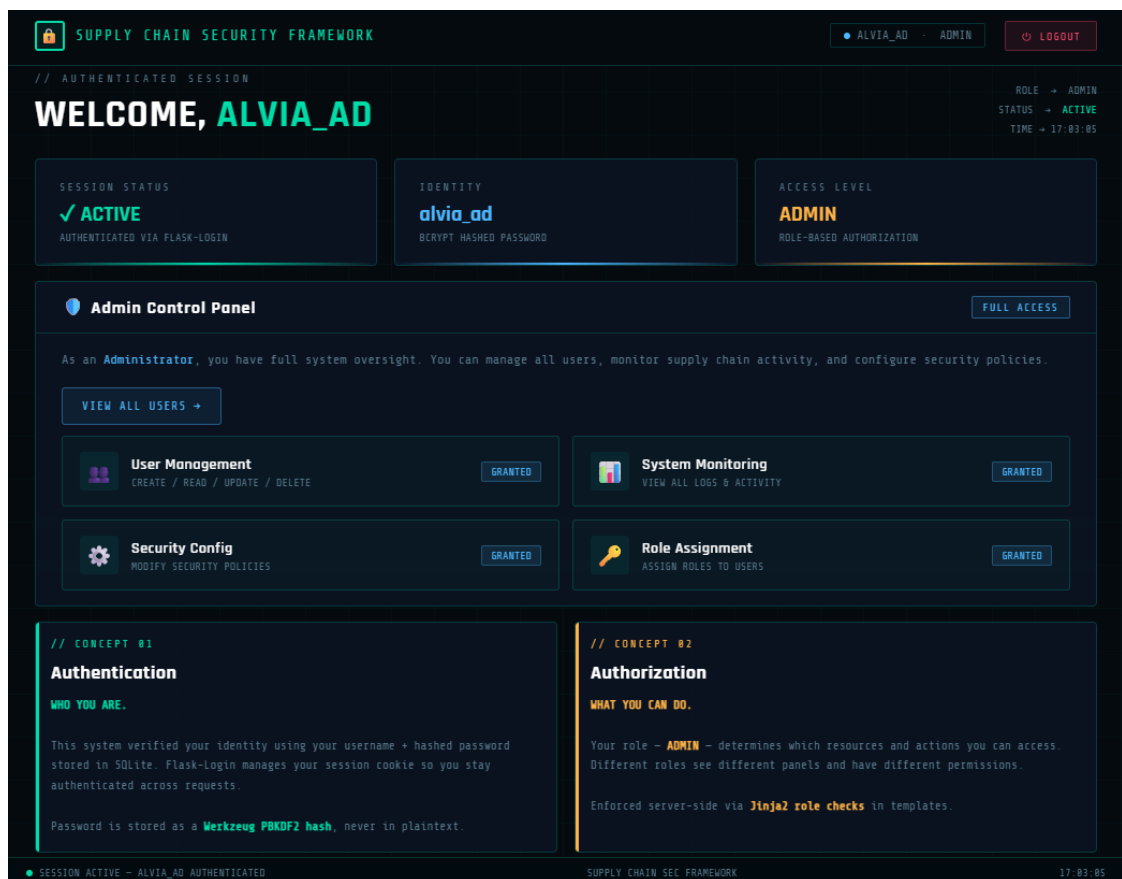
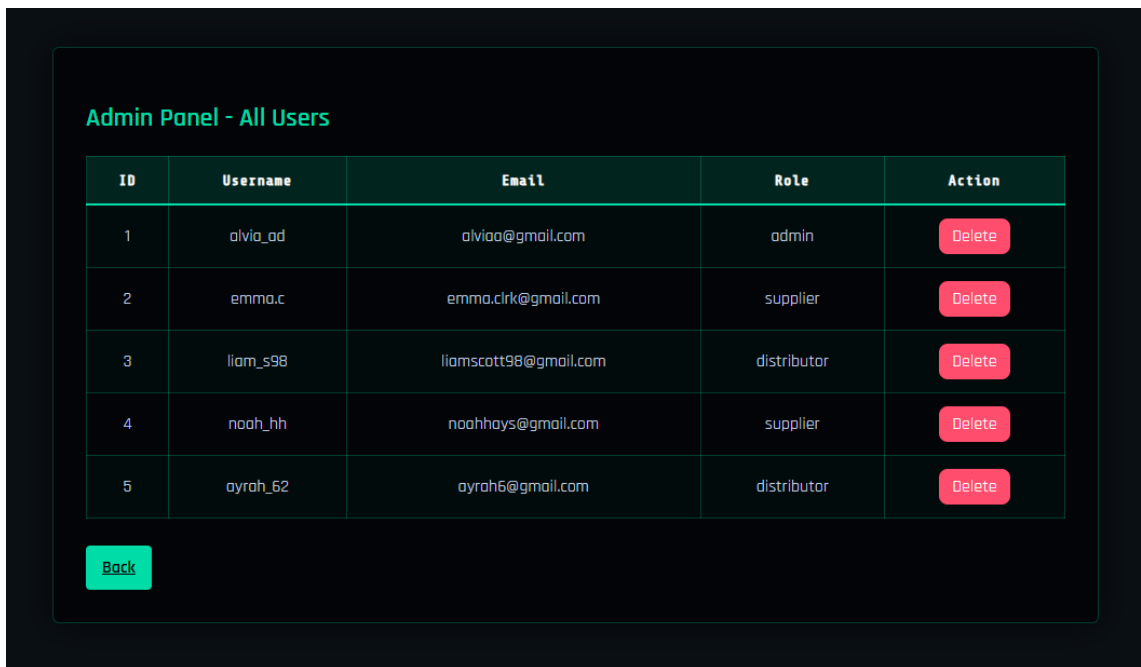


Figure 3: Dashboard — Authenticated User View

Admin Panel (admin.html):

The admin panel displays a table of all registered users showing ID, username, email, and role. Each row has a Delete button. The table uses teal-tinted column headers and alternating row shading. A Back button navigates to the dashboard. A self-deletion safeguard prevents the admin from deleting their own account. Flash messages for both self-deletion errors and successful deletions appear above the table.



Admin Panel - All Users

ID	Username	Email	Role	Action
1	alvia_ad	alviaa@gmail.com	admin	Delete
2	emma.c	emma.clrk@gmail.com	supplier	Delete
3	liam_s98	liamscott98@gmail.com	distributor	Delete
4	noah_hh	noahhays@gmail.com	supplier	Delete
5	ayrah_62	ayrah5@gmail.com	distributor	Delete

Back

Figure 4: Admin Panel — admin.html

9.2 Customer Testing

The application was tested manually using Google Chrome with the Flask development server running on localhost:5000. Both Black Box and White Box testing techniques were applied.

Black Box testing validated system behavior based on expected inputs and outputs without knowledge of the internal code. White Box testing verified the internal logic of individual route handlers and validation functions.

Testing was performed at three levels: Unit testing on individual components (registration validation, password hashing, role checks), Integration testing verifying data flow between components, and System testing evaluating the complete application end-to-end.

The full system test results are shown below:

Test Case	Input	Expected Result	Status
Valid registration	All fields valid, role = supplier	Account created, redirect to login	Pass
Empty field on signup	First name left empty	Flash error: All fields required	Pass
Short username	Username = ab	Flash error: min 4 chars	Pass
Short password	Password = abc	Flash error: min 6 chars	Pass
Duplicate username	Username already exists	Flash error: Username already exists	Pass
Duplicate email	Email already registered	Flash error: Email already registered	Pass
Valid login	Correct credentials	Redirect to dashboard	Pass
Invalid login	Wrong password	Flash error: Invalid username or password	Pass
Unknown username	Non-existent username	Same error as wrong password	Pass
Access dashboard — no session	No cookie	Redirect to /login	Pass
Access /admin as Admin	Admin role	Admin panel displayed	Pass

Delete another user	Admin deletes Supplier	User removed from database	Pass
Self-deletion attempt	Admin deletes own account	Flash error: cannot delete yourself	Pass
Logout	Click logout link	Session cleared, redirect to login	Pass

All 14 test cases passed successfully. No critical defects were identified during testing.

Flash Error Message — Invalid Login Attempt:

The screenshot below shows the flash error message that appears when a user enters an incorrect password on the login page.

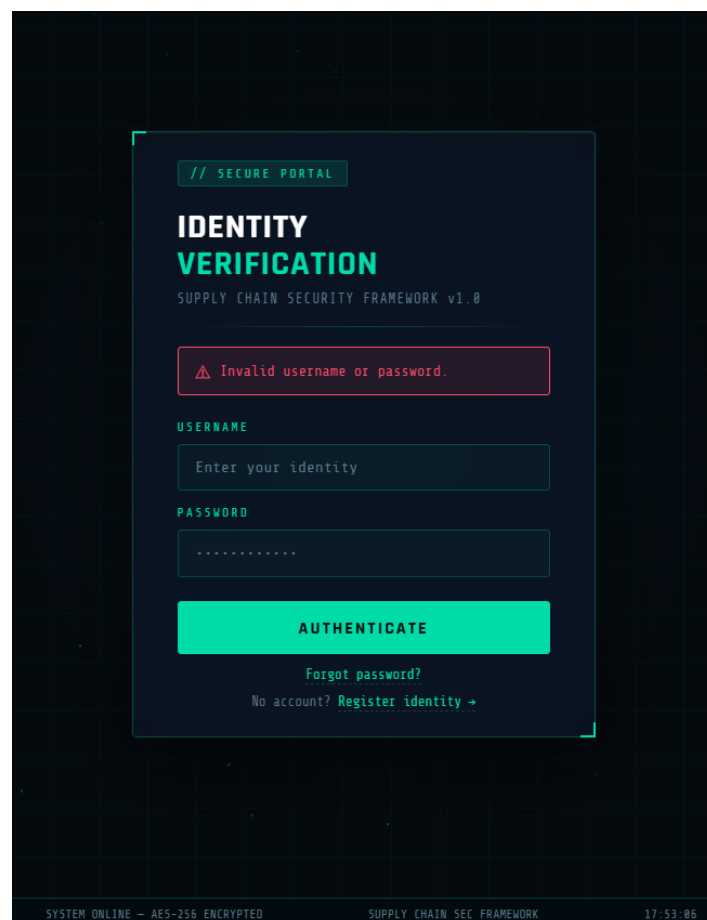


Figure 5: Flash Error Message — Invalid Login Attempt

Unit Testing Details

Unit testing was performed on each individual component of the application in isolation. The registration form validation was tested by submitting one invalid input at a time while keeping all other inputs valid. This confirmed that each of the seven validation checks operates independently and correctly a short username triggers only the username length error, a duplicate email triggers only the email uniqueness error, and so on. The password hashing mechanism was verified by registering a test account and then inspecting the database.db file using DB Browser for SQLite, confirming that the password column contained a Werkzeug PBKDF2 hash string and never any plain text value.

Integration Testing Details

Integration testing verified that data flows correctly between the components of the system. The registration-to-login flow was tested by registering a new account and immediately logging in with the same credentials, confirming that the hashed password stored during registration is correctly verified during login. The login-to-dashboard flow was tested for all three roles, confirming that the correct role-specific dashboard content is displayed based on the role stored in the database. The delete-to-admin-panel flow was tested by deleting a user and confirming that the user table on the admin panel refreshes to show one fewer row with the correct success flash message.

Black Box vs White Box Testing

Black Box testing treated the application as an opaque system and tested it purely through its user interface filling in forms, clicking buttons, and observing the results. This approach validated that the system behaves correctly from the user's perspective without any knowledge of the internal code. White Box testing, by contrast, involved reading the source code of each route handler and designing test cases that specifically

target the internal logic — for example, testing the self-deletion guard by directly clicking the delete button on the admin's own row, which tests the specific code path if `user` and `user.id == current_user.id`. The username enumeration prevention was tested by comparing the response messages for an unknown username versus a wrong password, confirming they are identical.

10. EVALUATION

10.1 Achievement of Objectives

The project successfully achieved all five of its defined objectives. The secure registration system was implemented with full server-side validation and Werkzeug password hashing. The credential-based login system correctly verifies passwords against stored hashes and creates signed session cookies on success. Role-based access control correctly distinguishes between Admin, Supplier, and Distributor users and enforces access boundaries server-side. The logout mechanism correctly terminates sessions. The system demonstrates the difference between Authentication and Authorization through its structure and dashboard content.

10.2 Strengths of the Implementation

The implementation has several notable strengths that reflect good software engineering and security practice:

- Password security is handled correctly — PBKDF2-HMAC-SHA256 with a random salt is an industry-standard approach, and no plain text passwords exist anywhere in the system.
- The username enumeration prevention design demonstrates awareness of a real-world attack vector and correctly addresses it through consistent error messaging.
- The self-deletion guard shows careful attention to edge cases that are easy to overlook but important for administrative systems.
- The dual-level uniqueness enforcement (application-level query check and database-level UNIQUE constraint) provides defense in depth at the data validation layer.
- The codebase is clean, well-structured, and separated into model, logic, and template layers, making it maintainable and easy to extend.

- The role-specific dashboard delivers genuinely different content based on the user's role, making the authorization model immediately visible and experiential rather than just conceptual.
- The consistent cybersecurity-themed UI reinforces the purpose of the system and creates a professional, coherent visual experience across all four pages.

10.3 Limitations

The system has several limitations that define the scope for future improvement:

- Role assignment is self-selected during registration. In a real-world supply chain system, roles would be assigned by an administrator after vetting the user, not chosen freely by the registering individual.
- The application runs on SQLite and Flask's development server, neither of which is suitable for production use with concurrent users or internet exposure.
- Email verification during registration is not implemented. Any email address, valid or otherwise, can be used to register.
- Password recovery is not implemented. A user who forgets their password cannot regain access without admin intervention.
- The admin panel does not implement pagination. `User.query.all()` retrieves all records at once, which would be inefficient for a large user base.
- Session cookies are not configured with the Secure and HttpOnly flags, which would be required in a production HTTPS deployment to prevent JavaScript access and transmission over unencrypted connections.
- No audit logging is implemented. Administrative actions such as user deletion are not recorded for compliance or security review purposes.
- The admin panel is only accessible to Admin users and is not linked from the Supplier or Distributor dashboard, which means role-based access control is demonstrated through the role-specific dashboard content rather than through an explicit access denied mechanism.

10.4 Comparison with Industry Standards

The security practices implemented in this project align closely with industry standards and best practices for web application identity management. The OWASP (Open Web Application Security Project) Authentication Cheat Sheet recommends storing passwords using adaptive one-way hashing functions with a work factor, which is exactly what Werkzeug's PBKDF2 implementation provides. The OWASP Top Ten list identifies Broken Authentication as one of the most critical web application security risks, and this project directly addresses this risk by implementing correct password hashing, session management, and account enumeration prevention.

The username enumeration prevention in the login route returning the same error message whether the username does not exist or the password is incorrect is a specific recommendation in the OWASP Authentication Cheat Sheet. This prevents an attacker from using the login form to discover which usernames are registered in the system, which is a common first step in credential stuffing attacks.

The dual-level uniqueness enforcement checking for duplicate usernames and emails both at the application level and the database constraint level provides defense in depth, which is a fundamental principle of secure system design. Defense in depth means that no single security control is relied upon exclusively; instead, multiple overlapping controls are used so that if one fails, another catches the vulnerability.

10.5 Learning Outcomes

This project provided significant practical learning across several areas of web application development and cybersecurity. The implementation of password hashing from scratch understanding why plain text storage is dangerous, how salting prevents rainbow table attacks, and how PBKDF2's iteration count makes brute force attacks computationally expensive provided a deep understanding of password security that goes beyond theoretical knowledge.

Working with Flask-Login provided a practical understanding of how HTTP sessions work, why HTTP is stateless and how cookies solve the session persistence problem, and how cryptographic signing protects session data from client-side tampering. Building the role-based access control system reinforced the fundamental distinction between authentication and authorization and demonstrated why both layers of control are necessary authenticating a user proves their identity, but does not by itself determine what they are allowed to do.

The project also provided experience with the full stack of a web application from the database schema and ORM layer through the business logic in Python to the HTML templates and CSS styling in the browser giving a holistic view of how a web application is structured and how data flows through each layer.

11. SNAPSHOTS OF THE PROJECT

This section presents full screenshots of the application across all major pages and user interactions.

Login Page – Initial State:

The login page with the full status bar visible at the bottom showing SYSTEM ONLINE — AES-256 ENCRYPTED.

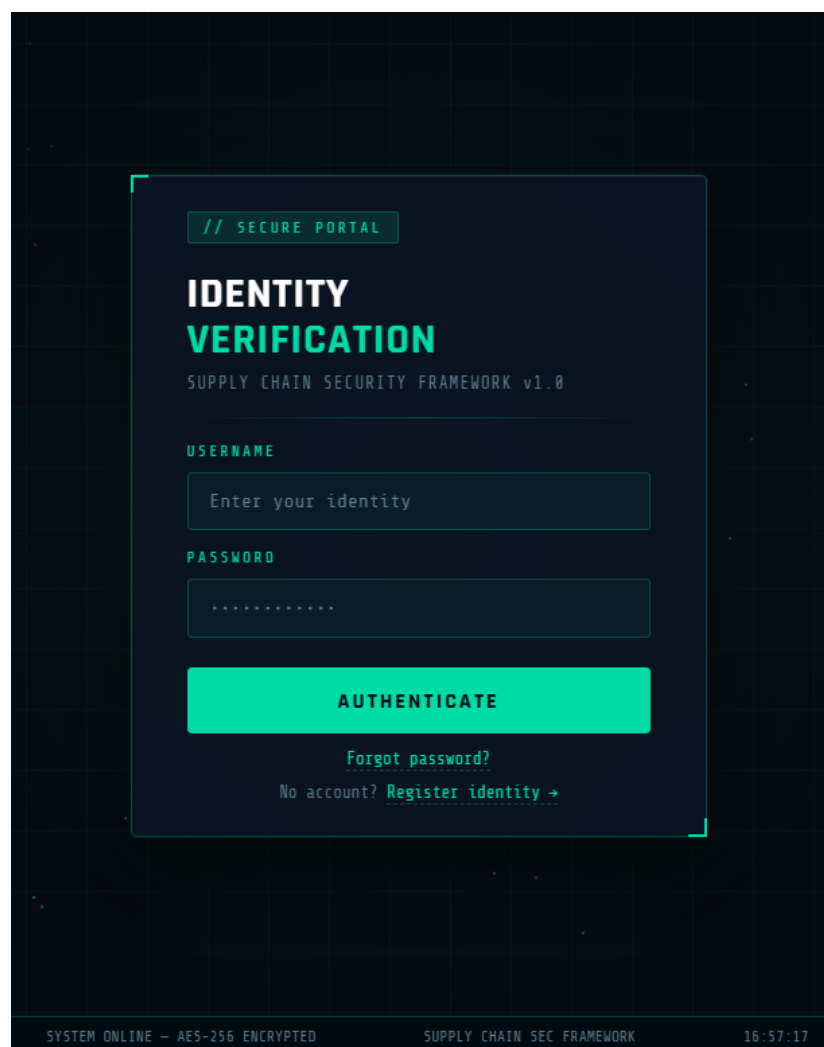


Figure 6: Login Page — Initial State

Signup Page – Role Selection Cards:

The signup page showing the three styled role selection radio button cards. The Supplier card is selected showing the teal highlight border.

// NEW IDENTITY

REGISTER ACCESS

CREATE YOUR SUPPLY CHAIN IDENTITY

FIRST NAME LAST NAME

Ayrah Khan

EMAIL ADDRESS

ayrah6@gmail.com

USERNAME

ayrah_62

PASSWORD

.....

ASSIGN ROLE

ADMIN SUPPLIER DISTRIBUTOR

REGISTER IDENTITY

Already registered? [Authenticate +](#)

● SYSTEM ONLINE – BCRYPT HASHING ACTIVE SUPPLY CHAIN SEC FRAMEWORK 17:10:43

Figure 7: Signup Page — Role Selection Cards

Signup Page — Validation Error Flash Message:

The signup page showing a validation error flash message when the username is too short.

The screenshot displays a dark-themed registration form titled "REGISTER ACCESS" with the subtitle "CREATE YOUR SUPPLY CHAIN IDENTITY". A red-bordered error message at the top states: "Username must be at least 4 characters." The form includes input fields for "FIRST NAME", "LAST NAME", "EMAIL ADDRESS", "USERNAME", and "PASSWORD". Below these is an "ASSIGN ROLE" section with three buttons: "ADMIN" (with a shield icon), "SUPPLIER" (with a cube icon), and "DISTRIBUTOR" (with a truck icon). A large green "REGISTER IDENTITY" button is at the bottom, followed by a link: "Already registered? [Authenticate →](#)". The footer contains the text "SYSTEM ONLINE - BCRYPT HASHING ACTIVE", "SUPPLY CHAIN SEC FRAMEWORK", and the timestamp "17:14:31".

Figure 8: Signup Page — Validation Error Flash Message

Dashboard — Supplier Role (No Admin Link):

The dashboard as seen by a Supplier user — no admin panel link is visible.

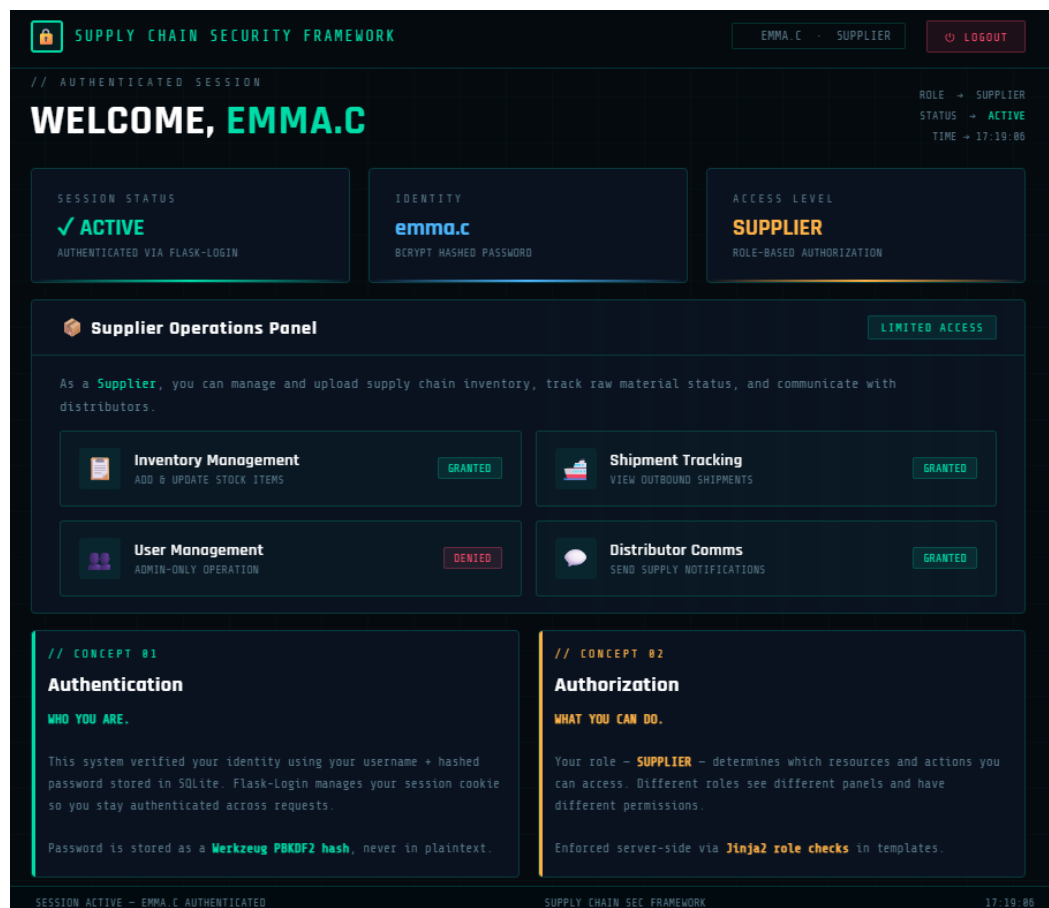


Figure 9: Dashboard — Supplier Role (No Admin Link)

Dashboard — Admin Role (Admin Panel Link Visible):

The dashboard as seen by an Admin user — the admin panel link is visible.

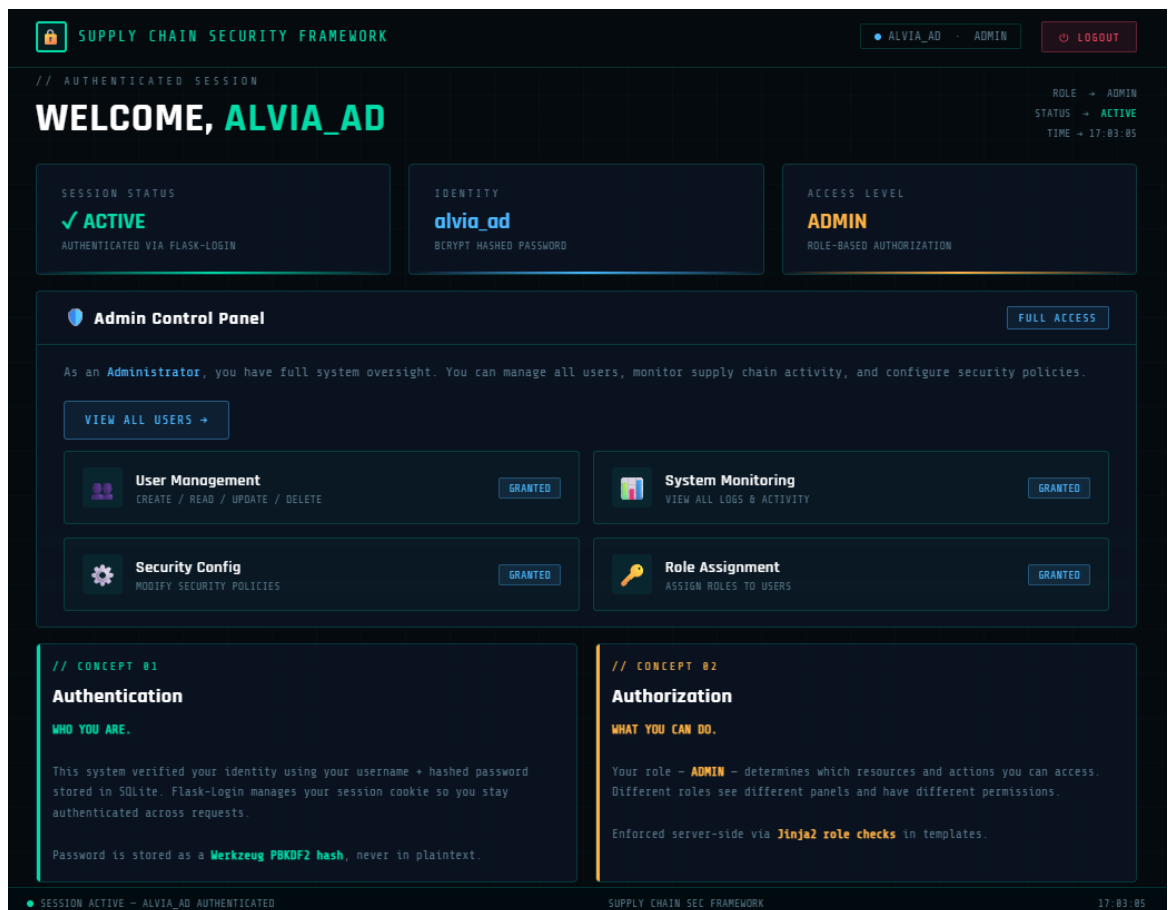
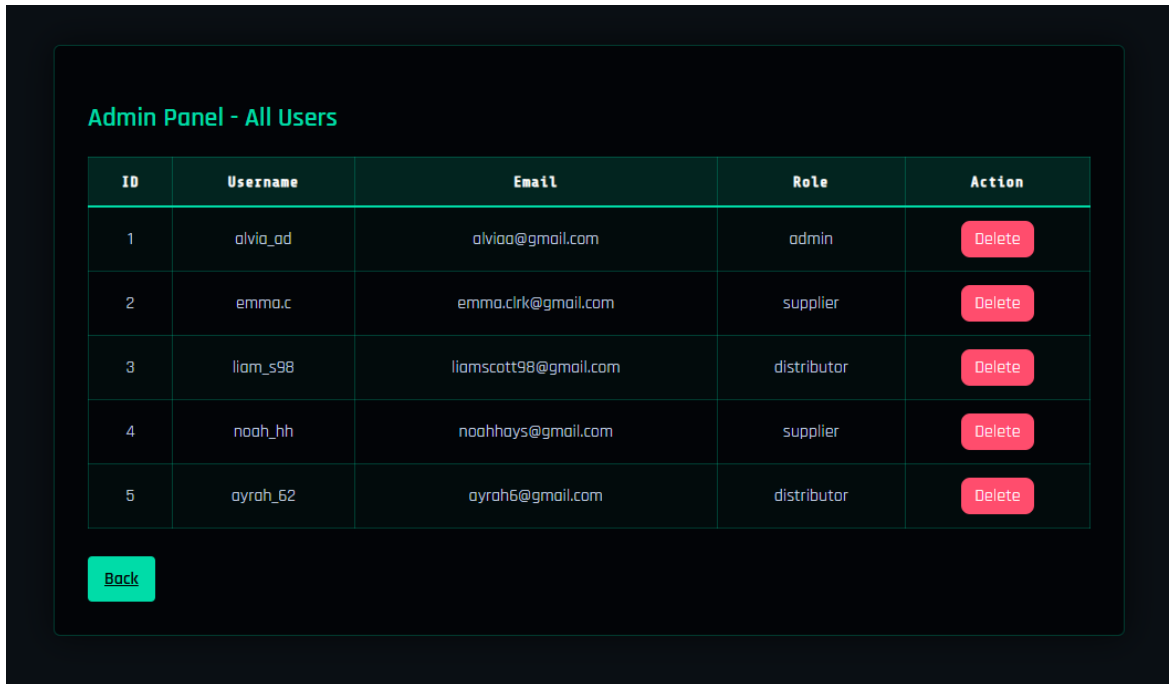


Figure 10: Dashboard — Admin Role (Admin Panel Link Visible)

Admin Panel — Full User Table:

The admin panel showing the full registered users table with delete buttons.



Admin Panel - All Users

ID	Username	Email	Role	Action
1	alvia_ad	alviaa@gmail.com	admin	Delete
2	emma.c	emma.clrk@gmail.com	supplier	Delete
3	liam_s98	liamscott98@gmail.com	distributor	Delete
4	noah_hh	noahhays@gmail.com	supplier	Delete
5	ayrah_62	ayrah6@gmail.com	distributor	Delete

Back

Figure 11: Admin Panel — Full User Table

Admin Panel — Self-Deletion Error Message:

The admin panel showing the error flash message when admin tries to delete their own account.

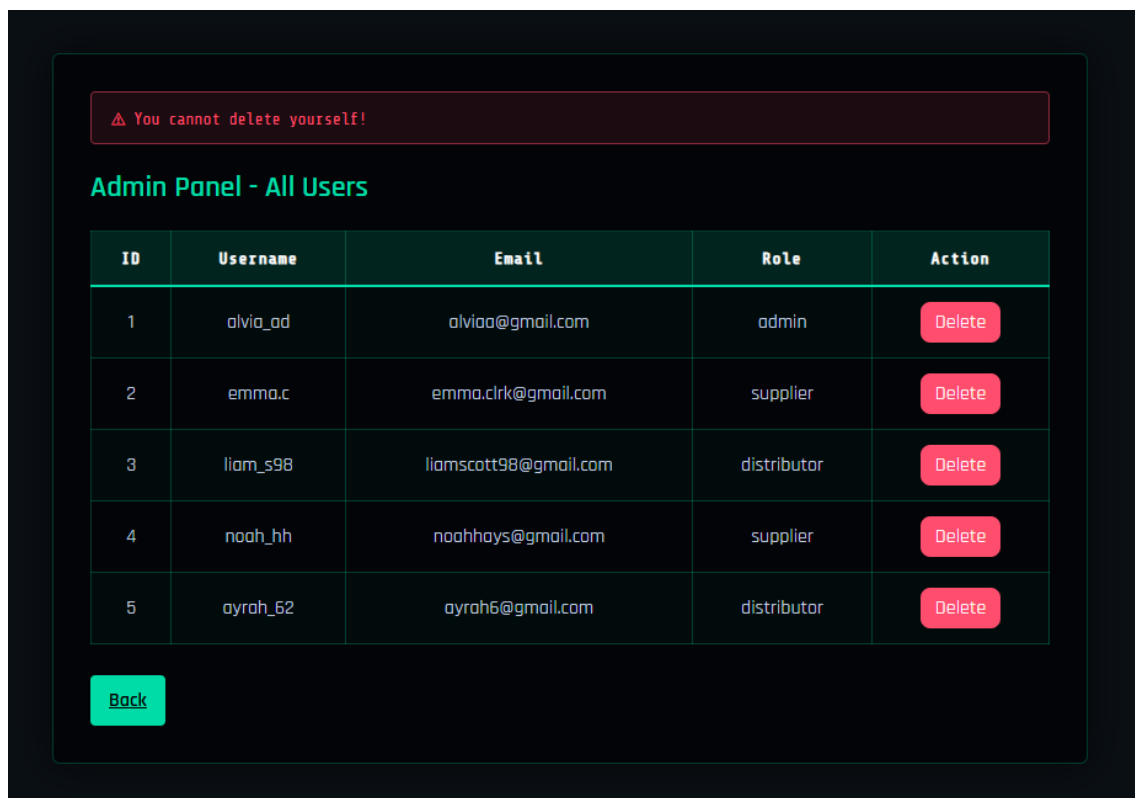


Figure 12: Admin Panel — Self-Deletion Error Message

Admin Panel — Successful User Deletion Message:

The admin panel showing the success flash message after a user has been deleted.

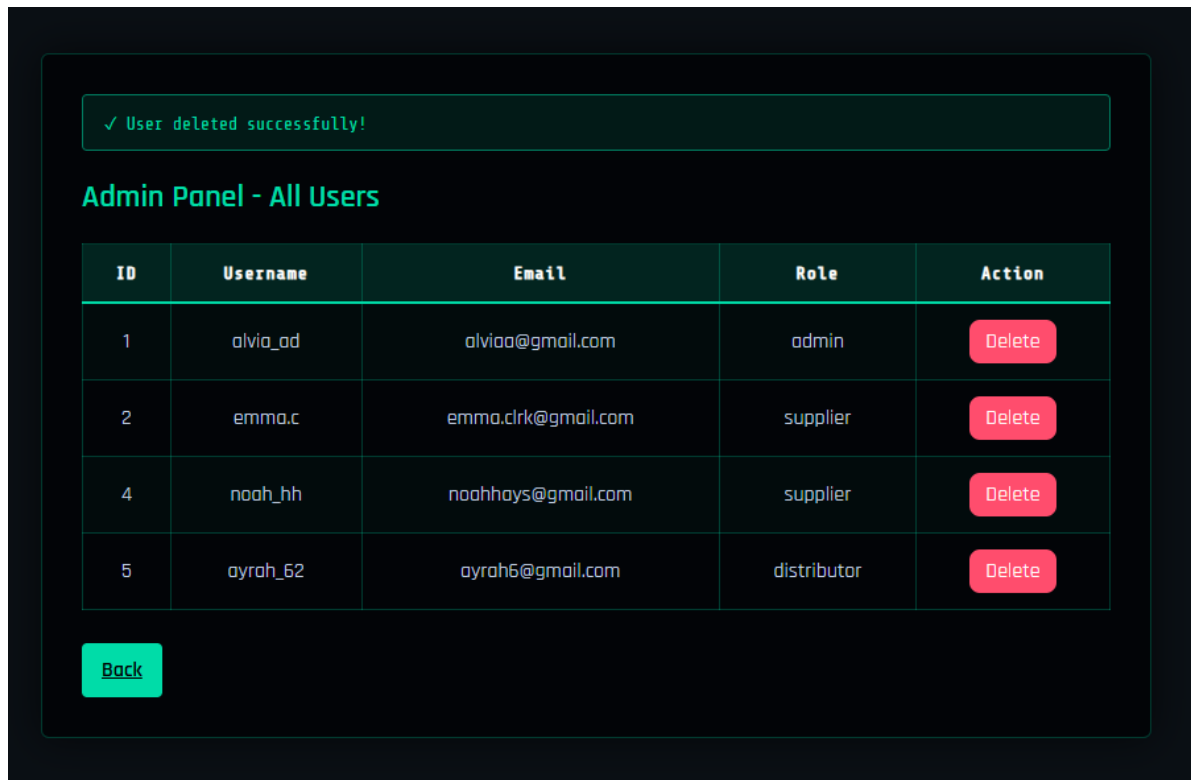


Figure 13: Admin Panel — Successful User Deletion Message

12. CONCLUSIONS

This project successfully demonstrates the implementation of a secure identity management system for a supply chain web application using Python, Flask, Flask-Login, and SQLite. The system covers the two foundational cybersecurity concepts that motivated its development — Authentication and Authorization — and implements them correctly using industry-standard tools and practices.

The development process involved defining clear requirements for a three-role supply chain authentication system, designing a relational database schema and seven-route application structure, building a Flask application with server-side validation and PBKDF2 password hashing, integrating Flask-Login for session management across all protected routes, enforcing role-based access control that delivers genuinely different dashboard experiences to Admin, Supplier, and Distributor users, and developing a consistent and professional cybersecurity-themed frontend interface. The system was tested thoroughly through unit, integration, and system testing methodologies, and all fourteen defined test cases passed successfully with no critical defects identified.

The key technical achievements of the project are the correct implementation of PBKDF2-HMAC-SHA256 password hashing using Werkzeug, the use of HMAC-signed session cookies for tamper-proof state management, the two-layer access control model that combines `@login_required` with role checks, the username enumeration prevention design in the login route, the self-deletion guard in the admin delete route, and the dual-level uniqueness enforcement for username and email fields. Each of these design decisions reflects real-world security patterns that are directly applicable to professional web development.

From a visual and user experience perspective, the project demonstrates that security-conscious design does not require sacrificing usability or aesthetics. The cybersecurity-themed UI with its animated backgrounds, role-specific dashboards, and clear visual indicators — including the GRANTED and DENIED badges on the role-specific feature cards makes the authorization model immediately visible and understandable to any user who interacts with the system.

From an educational perspective, the project achieves its primary goal of making abstract cybersecurity concepts tangible and experiential. A user logging in as a Supplier immediately sees what they can and cannot do based on their role. A user logging in as an Admin sees the full scope of their elevated access. The dashboard content itself becomes a live demonstration of Authorization in action — the same application, the same login process, but a fundamentally different experience based on the identity and role of the authenticated user. This is precisely what the project set out to demonstrate, and it does so clearly and effectively.

13. FURTHER DEVELOPMENT OR RESEARCH

Several enhancements could extend this project in future versions to bring it closer to a production-ready identity management system:

- **Email Verification:** Sending a verification email to the submitted address using Flask-Mail before activating the account. This would prevent registration with invalid or others' email addresses and add a second verification step to the registration process.
- **Password Recovery:** A forgot password flow using time-limited secure tokens (Flask's itsdangerous library) sent by email to the registered address. The token would be valid for a short window (15–30 minutes) and single-use to prevent replay attacks.
- **Invitation-Based Role Assignment:** Rather than allowing users to self-select their role during registration, an admin would send invitation links with pre-assigned roles. This would prevent unauthorized role selection and align the system with how real supply chain identity management works in organizations.
- **Multi-Factor Authentication:** A second authentication factor such as a time-based one-time password (TOTP) from an authenticator app.
- **Production Deployment:** Replacing SQLite with PostgreSQL (changing a single configuration line), serving through a multi-worker WSGI server such as Gunicorn, configuring HTTPS with a TLS certificate, and enabling the Secure and HttpOnly flags on session cookies. The application architecture was designed to support this upgrade path with minimal code changes.

- **Audit Logging:** Recording all login attempts, role changes, and deletions to a separate audit log table for compliance purposes.
- **REST API with JWT:** Exposing authentication as a REST API with JWT tokens for mobile or third-party service to authenticate against the same identity framework.
- **Admin Role Management Interface:** Adding an interface within the admin panel that allows administrators to change a user's role after registration, rather than relying on the role set during signup. This would make the system more operationally realistic.

14. REFERENCES

- Flask Official Documentation. Pallets Projects. <https://flask.palletsprojects.com>
- Flask-Login Documention. Matthew Frazier. <https://flask-login.readthedocs.io>
- Flask-SQLAlchemy Documentation. Pallets Projects. <https://flask-sqlalchemy.palletsprojects.com>
- Werkzeug Documentation. Pallets Projects. <https://werkzeug.palletsprojects.com>
- SQLite Official Documentation. <https://www.sqlite.org/docs.html>
- Python 3 Official Documentation. Python Software Foundation. <https://docs.python.org/3/>
- OWASP Authentication Cheat Sheet. OWASP Foundation. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Top Ten Web Security Risks. OWASP Foundation. <https://owasp.org/www-project-top-ten/>
- OWASP Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Mozilla Developer Network. HTTP Cookies. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

15. APPENDIX

Appendix A — Installation and Setup Instructions

Follow these steps to set up and run the Identity Security Framework for Supply Chain on any machine with Python 3.x installed:

- Step 1: Install Python 3.x from <https://www.python.org/downloads/> if not already installed.
- Step 2: Open a terminal and navigate to the project folder.
- Step 3: Install required packages: `pip install flask flask-login flask-sqlalchemy`
- Step 4: Run the application: `python app.py`
- Step 5: Open a browser and navigate to: `http://localhost:5000`
- The application will automatically create the `database.db` file on first startup. No manual database setup is required.

Appendix B — `models.py` Full Source Code

```
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin

db = SQLAlchemy()

class User(UserMixin, db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    first_name = db.Column(db.String(100), nullable=False)
    last_name = db.Column(db.String(100), nullable=False)
```

```
email = db.Column(db.String(150), unique=True, nullable=False)
```

```
username = db.Column(db.String(100), unique=True, nullable=False)
```

```
password = db.Column(db.String(200), nullable=False)
```

```
role = db.Column(db.String(50), nullable=False)
```

Appendix C — Key app.py Configuration

```
app.config['SECRET_KEY'] = 'change-this-in-production'
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
login_manager.login_view = 'login'
```